

目录

- AI Loop 工程框架对比分析报告
 - Claude Code / Codex / Superpowers / grill-me / Auto-engineering / LangGraph+Deep Agents 六方对比
 - 一、六个对比对象
 - 二、Auto-engineering 项目发展史
 - 2.1 版本演进时间轴
 - 2.2 关键转折点: v5.0 失败与 v5.6 重建
 - 2.3 自我推翻的决策 (证明项目是踩坑迭代而非先知预言)
 - 2.4 明示的借鉴清单
 - 2.5 明示的禁用清单
 - 2.6 两个需要正视的担忧
 - 2.7 项目发展史的核心启示
 - 三、六方统一对比大表
 - 四、范式定位图
 - 五、Auto-engineering vs LangGraph + Deep Agents 的本质区别
 - 三句话总结
 - 六、10 项能力覆盖矩阵 (六方)
 - 七、对银行的判断
 - 7.1 六方对银行场景的适用性
 - 7.2 LangGraph + Deep Agents 对银行的核心价值
 - 7.3 Auto-engineering 的真实定位
 - 7.4 银行落地路径
 - 7.5 分层组合 > 单体框架
 - 参考来源
 - LangGraph 相关
 - Deep Agents 相关
 - 框架对比相关
 - Claude Code / Codex /goal 相关
 - Superpowers / grill-me 相关

AI Loop 工程框架对比分析报告

Claude Code / Codex / Superpowers / grill-me / Auto-engineering / LangGraph+Deep Agents 六方对比

读者对象: 银行软件开发工程师 (对 AI Loop 工程不甚熟悉)
报告范围: 六个 AI Loop 工程对象的范式定位、能力覆盖、Auto-engineering 项目发展史、银行落地可行性

一、六个对比对象

本报告对比的六个对象, 按"控制流位置"分为两大范式:

#	对象	形态	范式
1	Claude Code (含 /goal、/loop slash command)	官方 CLI agent + slash command	说服式
2	Codex (含 /goal Ralph Loop)	官方 CLI agent + slash command	说服式
3	Superpowers	9 个 SKILL.md 纯 markdown plugin	说服式
4	grill-me 1.1	1 个 SKILL.md (3 句话) + /grill-with-docs 扩展	说服式
5	Auto-engineering	Python Engine + 三平台 plugin	强制式 (垂直 loop)
6	LangGraph + Deep Agents	LangGraph (运行时) + Deep Agents (harness, 26.3K★)	强制式 (通用 harness)

需要先澄清一个关键认知: **LangGraph 和 Deep Agents 是同一战线的两层**——LangGraph 是底层编排运行时 (Pregel 超步 + checkpoint), Deep Agents 是 LangChain 官方在 LangGraph 之上构建的"开箱即用 agent harness"。本报告将两者合并为第六个对比方。

"说服式"与"强制式"的范式分界是本报告的核心分析框架:

- 说服式: 控制流在 LLM 脑内, 靠 prompt 说服 LLM 按流程走, agent 可被"压力/疲劳"说服绕过纪律
- 强制式: 控制流在 Python 进程内, 用代码强制 LLM 必须按流程走, Python 不看 agent 心情

二、Auto-engineering 项目发展史

在展开六方对比前, 需要先讲清 Auto-engineering 这个对象的来龙去脉。它不是一夜写成的, 而是一个历经 6 个大版本、65 条 BEACON 决策、多次自我推翻的演进产物。理解它的发展史, 才能理解它为什么是今天这个样子, 也才能理解它和 LangGraph + Deep Agents 的真正关系。

2.1 版本演进时间轴

基于项目 [design/INDEX.md](#) 合并日志和 [design/BEACON.md](#) 决策表, Auto-engineering 的演进可划分为 6 个阶段:

阶段	时间	版本	关键事件	BEACON 决策
初版	2026-06-24~28	v1.0 → v2.5	LoopEngine/StageGraph/AgentRuntime → Channel/TaskDAG/ConvergenceJudge → Gates/CLI → v1.0 退役	#1-28

阶段	时间	版本	关键事件	BEACON 决策
合订与拆分	2026-06-29~30	v5.0	Plugin + Loop + Init 合订，参考 LangGraph/AutoGen/CrewAI；随即 Init 拆分独立项目	#29-30
深度审计与失败	2026-07-04~06	v5.0→v5.5	4 P0 修复； Agent 自调节执行模型被判定为 v5.0 失败根因 ；v5.5 DeepAudit 扩展	#31, #38
范式重建	2026-07-08	v5.6	Tick-Based Discrete Invocation 协议确立，JSONL 协议被废弃；5 层验证架构；Gap Analysis	#39-42
方法论内化	2026-07-08~12	v5.6+	借鉴 Superpowers 提示词技术 + 验证方法论；外部 Skill/Agent 依赖审计；Commit→PR→CI/CD 分层	#43-52
架构收束与多平台	2026-07-12~17	v7.0/v8.0	单引擎 + 双驱动架构（v7.0）；多 Agent 平台适配（v8.0，三平台一套源码）；StandaloneDriver 基准修复	#54-56

2.2 关键转折点：v5.0 失败与 v5.6 重建

整个项目最关键的转折发生在 2026-07-04~08 之间。

v5.0 的失败根因：v5.0 采用"Agent 自调节执行模型"——通过有说服力的提示词让 Agent 自愿遵守规范（这正是 Superpowers 的核心机制）。但实测发现 Agent 会被"压力/疲劳"说服绕过纪律，导致生产失效。BEACON 决策 #43 明确记录：“Agent 自调节 = v5.0 灭亡根因”。

v5.6 的重建：决策 #39 确立 Tick-Based Discrete Invocation 协议——把连续的 while 循环替换为离散 CLI 调用（`ae dev-loop --init` → `--tick --result loop`），Python 每次 tick 独立进程，**Python 永不调 LLM API**，Agent 通过反复调用 `--tick` 驱动循环。这是一个范式级反转：从"信任 Agent 自觉"转向"用 Python 代码强制纪律"。

2.3 自我推翻的决策（证明项目是踩坑迭代而非先知预言）

BEACON 决策表里有多条"❌ superseded"，说明作者走过弯路：

决策 #	内容	状态	被推翻原因
#32	Agent Tool spec 模式	❌ 撤销	Agent 可能跳过规范，markdown 规则无法强制执行 → 改为 JSONL
#33	Agent-Engine JSONL 通信协议	❌ superseded by #39	v5.6 改为 Tick 文件桥接协议
#34	AE_JSONL_MODE 条件开关	❌ superseded by #39	v5.6 Tick 协议无需条件开关
#36	TDDGate + StageTransitionGate	❌ superseded	两者实现的是有状态 Guardrail 检查而非无状态 Gate，与 Gate.run() 接口不兼容

2.4 明示的借鉴清单

基于 [v5.6-Design-Loop.md](#)、[BEACON.md](#) 和 [discussion/v5.6-layered-verification-design.md](#) 的明示记录，Auto-engineering 借鉴了以下成熟项目：

借鉴对象	借了什么	用在哪	证据
LangGraph	PregelLoop 的 BSP tick/after_tick 控制流； <code>recursion_limit</code> 收敛模式	v5.6 整个 Tick-Based 架构的骨架	决策 #29、#39
CrewAI	Guardrail post-filter；GuardrailResult 三态（ <code>success/result/error</code> ）	9 道 Guardrail 的 pass/block/retry 三态	决策 #36
AutoGen	FIFO message switch（消息交换机）	Agent 间消息流设计	决策 #29
Superpowers	Iron Law / Red Flags / 合理化破解表 / CSO description 纪律	注入 9 个角色 prompt 的 fragments	决策 #43(B11)、#47(B15)
SonarQube	条件门禁模式	Gate 的 severity 阈值判定	决策 #36
Copier	Worker 上下文管理器（ <code>__enter__/_exit__</code> ）	Init-engineering 的 InitWorker	Init 侧
Cookiecutter	进度条	Init 交互式问答	Init 侧
Yeoman	composable routes 组合模式	Init 模板分层（ <code>_shared/_features/type</code> ）	Init 侧
VS Code extension manifest	插件清单格式	plugin.json 结构	决策 #29
Ports & Adapters（六边形架构）	端口适配器模式	v7.0 双驱动（Driver A 文件桥接 / Driver B Standalone）	决策 #54

2.5 明示的禁用清单

Auto-engineering 不仅明示借鉴了什么，还明示**不借鉴什么**和**禁用什么**。这是判断工程水平的关键——知道拒绝什么比知道借什么更重要。

不借鉴清单（来自 [discussion/v5.6-layered-verification-design.md](#) §十和决策 #43/#47）：

不借鉴对象	不借鉴内容	不借鉴理由
Superpowers	Agent 自调节执行模型	“其 Agent 自调节正是 v5.0 失败根因”。只用它的提示词技术，但把铁律转成 Python 确定性门控强制执行

不借鉴对象	不借鉴内容	不借鉴理由
Superpowers	subagent-per-task 编排	与 Tick loop 冲突
Superpowers	压力测试评估法	独立项目
Superpowers	阈值自学习	YAGNI（决策 #38 原 P1 阈值自学习被撤销）

禁用清单（来自决策 #46 外部 Skill/Agent 依赖审计）：

项目对 `auto_engineering/` 全量代码（75 个 .py + 3 个命令 + SKILL + 5 个 hook）做了运行时外部依赖审计，确立了“不 spawn 外部 agent、借鉴=复制非链接、系统依赖需抽象”三条准入规则。具体禁用项：

禁用对象	禁用理由
Claude Code 内置 Plan subagent	自有 architect role 替代
Claude Code 内置 code-reviewer subagent	自有 critic role + 5 层验证替代
gsd-* 系列 skill	零容忍——2026-07-04 生产失效根因
MCP 工具	零运行时调用
任何外部 agent spawn	自有 role + B12 prompt 替代

`skills/auto-engineering/SKILL.md` 开篇即写明：“This loop has **zero runtime external dependencies**. Do **not** spawn external agents (Plan, code-reviewer, gsd-*, MCP tools) as part of the loop.”——它在 Claude Code 之上故意禁用了内置 subagent，全用自己的 Python + role prompt 替代，原因就是要拿回控制权。

2.6 两个需要正视的担忧

在禁用内置 subagent 这条设计决策的背后，有两个值得银行工程师正视的担忧。

担忧一：上下文隔离缺失，长任务是否会撑爆上下文窗口？

担忧内容：Auto-engineering 禁用了 subagent（这是 Claude Code/Codex 原生的上下文隔离机制），9 个角色（architect/developer/critic + 6 个验证层）全部在同一个 agent 的同一个 context window 内通过 stage 切换实现。对短任务这没问题，但对长时间、大项目（银行典型场景：一个信贷系统改造涉及几十个模块、上千文件），累积的对话历史、工具调用结果、设计文档引用会迅速塞满 context window，导致 LLM 遗忘早期指令、混淆不同阶段信息——这正是 Deep Agents 文档里反复强调的“Agent 1.0 失败根因”。

项目已有的缓解措施（核实自代码）：

机制	位置	作用	局限
<code>_truncate_tool_results</code>	agents/base.py:36	截断 tool_result 到 8000 字符防爆炸	仅 v5.5 连续循环路径；治标不治本
<code>AE_MAX_TOOL_CALLS=10</code>	plugin.json:56	单次 agent 调用工具循环上限	限制单 tick 不限制总累积
Tick 独立进程	v5.6 协议核心	每次 tick 独立进程，SQLite 跨 tick 持久化状态	状态隔离了，但同一 tick 内仍是单 context
5 层验证裁剪（LEAF/PLATE/FULL）	决策 #41	按设计层次自动跳过不需要的验证层	减少角色数，不减少单角色上下文压力

核心判断：这些机制都是“截断/限制/裁剪”，没有 context offloading（把中间结果卸载到文件、需要时再读回）。Deep Agents 的 summarization + 文件 offloading + prompt caching 是系统性解法，Auto-engineering 缺这一层。对银行大项目，这是真实风险——长任务跑到中后期，agent 可能因 context 压力出现“幻觉漂移”（hallucination drift），而项目的 Python 门控只能校验产出格式，校验不了 agent 是否“想对了”。

担忧二：这个项目的最佳使用形式是什么？

项目实际形态（核实自 `plugin.json`、`skills/`、`commands/`、`hooks/`）：它是一个标准的 Claude Code Plugin，由四部分组成：

组成	路径	作用	是否必需
Plugin 清单	<code>.claude-plugin/plugin.json</code> (+ <code>.codex-plugin/</code> + <code>.codebuddy-plugin/</code> 三平台)	声明插件元信息、环境变量、依赖	必需（入口）
Skill	<code>skills/auto-engineering/SKILL.md</code>	Claude Code 情境匹配时自动加载的调度脚手架（含 Hard Constraints）	必需（让 agent 知道何时用、怎么用）
Slash Commands	<code>commands/*.md</code> （10 个命令：dev-loop/status/checkpoint/audit 等）	用户显式触发的命令入口	必需（主入口 /dev-loop）
Hooks	<code>hooks/*.sh</code> （5 个：session-start/pre-tool/post-edit/on-pr/stop）	生命周期事件钩子	可选（增强自动化）

所以它不是“作为技能”或“作为插件”二选一——它是 plugin，skill 和 command 是 plugin 内部的两种暴露形态：

- **Skill（被动触发）**：当用户说“实现一个功能”时，Claude Code 情境匹配到 SKILL.md 的 description，自动建议用 /dev-loop。Skill 负责告诉 agent “这个 loop 的铁律是什么”（Hard Constraints 7 条）。
- **Slash Command（主动触发）**：用户显式输入 /dev-loop “需求”，触发 `commands/dev-loop.md` 定义的 tick 协议，agent 按 protocol 反复调用 `ae dev-loop --tick`。
- **Hooks（生命周期）**：如 `stop.sh` 在 agent 想结束时拦截，强制检查是否真的跑完所有 Gate。

对银行的建议：

1. 不要试图只用其中一部分。SKILL.md 单独拿出来没用（它只是说明文档），commands 单独拿出来会失去 Hard Constraints 的 prompt 约束，hooks 单独拿出来只是空脚本。四件套必须整体安装为 plugin。

2. 三平台 plugin 清单是 v8.0 才加的（决策 #55），但 CodeBuddy 是 fallback 读 .claude-plugin/plugin.json，Codex 缺 on-pr.sh——跨平台行为并不完全一致，银行若选非 Claude Code 平台需自行验证。
3. 真正的问题不在"安装形式"，在"安装后 agent 是否听话"。SKILL.md 的 Hard Constraints 是 prompt 级说服，agent 仍可能绕过（这正是 v5.0 失败根因）。项目的真正保险是 Python 门控——只要 agent 敢跳过 ae dev-loop --tick 直接改代码，下次 --tick 会因 working tree 不干净被 Guardrail 拦截。所以使用形式的关键不是 skill/command/hooks 怎么配，而是必须保证 agent 走 tick 协议，不能旁路。

这两个担忧合起来指向同一个结论：**Auto-engineering 的禁用 subagent 决策是用"控制力"换"上下文隔离能力"**——它拿到了对 agent 行为的强控制，但放弃了 subagent 提供的 context 隔离。对短任务这是合理权衡，对银行大项目这是需要补的短板，而补法正是上一节提到的"构建在 Deep Agents 之上"——用 Deep Agents 的 sub-agent 隔离 + context offloading 补 Auto-engineering 的上下文短板，用 Auto-engineering 的 9 角色 + 7 Gate 纪律补 Deep Agents 的软件开发垂直能力。

2.7 项目发展史的核心启示

Auto-engineering 的演进史揭示了一个重要事实：它的**"复杂度"主要来自踩坑时间和试错成本，而非单点智力门槛。**

- 代码量大（2135 测试、几十个模块）——是 6 个大版本迭代累积的结果
- 设计文档厚（BEACON 65 条决策 + v5.6-Design-Loop 4000+ 行）——是踩坑记录，不是先知预言
- 概念多（Tick/Guardrail/Gate/Convergence/Checkpoint）——每个单独看都不难，难在组合后的状态机正确性，而这部分作者直接借鉴 LangGraph 的成熟模式

关键判断：作者最值钱的能力不是"发明新东西"，而是"知道借什么、知道拒绝什么"——把 LangGraph 的 tick 骨架 + CrewAI 的 guardrail + Superpowers 的提示词纪律 + 六边形架构的驱动抽象，整合成一个"Python 门控优先、LLM 只做执行"的闭环。这个整合判断力是核心价值，但它不是魔法，是可学习的工程决策模式。

三、六方统一对比大表

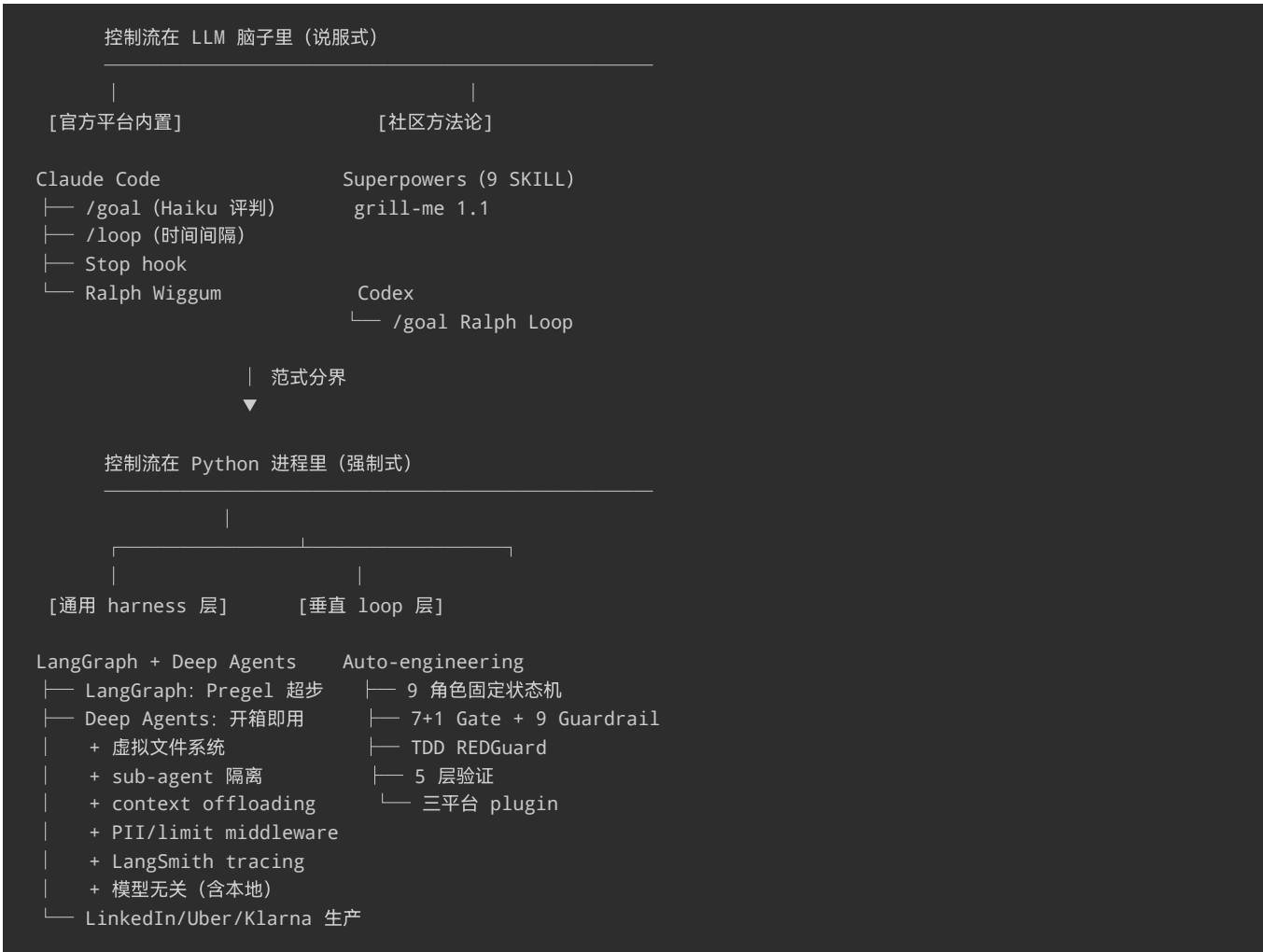
维度	Claude Code (含 /goal、/loop)	Codex (含 /goal)	Superpowers	grill-me 1.1	Auto-engineering	LangGraph + Deep Agents
形态	官方 CLI + slash command	官方 CLI + slash command	9 SKILL.md plugin	1 SKILL.md	Python Engine + 三平台 plugin	LangGraph (运行时) Deep Agents (harness 0.6.12 稳定 / 0.7 alpha 26.3K★)
控制流位置	LLM 脑内 (说服式) + Stop hook	LLM 脑内 + app-server 持久化	LLM 脑内 (说服式)	LLM 脑内 (说服式)	Python 进程 (强制式)	Python 进程 (graph 编排, LLM 只是被节点调用)
完成判定	Haiku 每轮判 yes/no	Ralph Loop 5 阶段自评	无显式判定	设计树全分支解决	4 级收敛	条件边 → END, 由开发定义 (可做收敛, 可做上限)
状态持久化	session-scoped	app-server 跨会话 resume	无	无	SQLite WAL	原生 Checkpointer (Sqlite/Postgres/Redis + time travel 回溯)
多角色协作	Agent Teams (Lead+Teammate 并行)	单 agent + 工具循环	subagent-per-task	单轮审问	9 角色固定状态机	supervisor / hierarchical collaborative 三模式 + sub-agent 隔离 context window
TDD 强制	说服式	说服式	会删先写代码	不涉及	G7 REDGuard 强制	无内置, 需自己加 middleware
代码审查	code-reviewer subagent	自我 Review 阶段	2 skill	不涉及	critic + 5 层验证	无内置, 需 supervisor pattern 自建
质量门禁	无	无	无	无	7+1 Gate + 9 Guardrail	middleware hook (modelCallLimit / toolCallLimit / PII / permissions)
跨平台	Claude Code 专属	Codex 专属	Claude/Codex/Cursor/Gemini/OpenCode	任何 markdown agent	Claude Code/Codex/CodeBuddy	平台无关 (Python 进程不挂任何 agent)
LLM 后端	Anthropic 专属	OpenAI 专属	同宿主	同宿主	Anthropic + OpenAI	任何 LangChain 兼容模型 (含 Ollama 本地、GLM、通义、文心)
token 成本控制	/goal Haiku 较省	budget_limited 软停	无	无	max_iterations + stagnation 熔断	modelCallLimit + toolCallLimit + prompt caching + modelFallback
可审计性	黑盒	app-server 留存	黑盒	黑盒	tick JSON + prompt sha256 锁	LangSmith 一等公民 tracing (每节点可观测)
上手成本	零 (内置)	零 (内置, 需 ChatGPT 订阅)	低	极低	高	LangGraph 高 (graph 概念); Deep Agents 中 (开箱即用但要懂 Python)
灵活性	极高	高	高	极高	低 (固定状态机)	极高 (任意 graph + 可 override 任意 piece)
认证门槛	Claude 订阅	必须 ChatGPT 订阅, API key 不行	Claude Code	任何 agent	自备 key	无 (自备 key, 可本地 Ollama)

维度	Claude Code (含 /goal、/loop)	Codex (含 /goal)	Superpowers	grill-me 1.1	Auto-engineering	LangGraph + Deep Agents
成熟度	官方维护	官方维护	社区头部	社区头部	个人项目，无 SLA	LangChain 官方维护 + LinkedIn/Uber/Klarna 产验证
用户量	Claude Code 全量	500 万+周活	94 万+	4.9 万+	个人项目	LangGraph 生产级；Deep Agents 26.3K★

大表关键洞察：

- 完成判定机制是分水岭：Claude Code /goal 和 Codex /goal 用"小模型评判"（轻量但靠 LLM 判断，可能误判）；Auto-engineering 用"Python 4 级收敛"（重但确定性）；LangGraph + Deep Agents 用"条件边"（开发者自定义，灵活度最高）；Superpowers/grill-me 没有显式判定，全靠 agent 自觉或用户喊停。
- 状态持久化是生产关键：Codex /goal 的 app-server 持久化是它最大的工程优势——能"跑过夜"；Auto-engineering 的 SQLite 是对银行场景的卖点——掉电可恢复；LangGraph + Deep Agents 的 Checkpointer + time travel 是生产级最强的——不仅能恢复，还能回溯到任意历史状态重放。
- 认证门槛差异：Codex /goal 强制 ChatGPT 订阅，API key 不可用——对企业自建代理是硬门槛；Claude Code /goal 只需 Claude 订阅；Auto-engineering 自备 key；LangGraph + Deep Agents 无认证门槛，可本地 Ollama 完全离线运行——这对银行内网部署是决定性优势。

四、范式定位图



这张图的核心信息：Auto-engineering 和 LangGraph + Deep Agents 在同范式（Python 强制式），但不同抽象层——前者是"软件开发垂直 loop"，后者是"通用 agent harness"。结合第二章的项目发展史可知：Auto-engineering 自建的 tick 编排、SQLite checkpoint、token 限流，与 LangGraph + Deep Agents 的成熟能力高度重叠。这意味着 Auto-engineering 完全可以构建在 Deep Agents 之上：9 角色变 9 个 sub-agent，7 Gate 变 middleware，SQLite 换成 StoreBackend。

五、Auto-engineering vs LangGraph + Deep Agents 的本质区别

两者同范式但不同层次，差异不在"谁更强"，在"解决什么问题"：

维度	Auto-engineering	LangGraph + Deep Agents
抽象层次	垂直 loop（软件开发 9 阶段）	通用 harness（任意 agent 任务）
设计哲学	opinionated 到死（9 角色固定）	opinionated 但可 override 任意 piece
多角色实现	同 agent 内 stage 切换	sub-agent 隔离 context window

维度	Auto-engineering	LangGraph + Deep Agents
上下文管理	tick 内全量 prompt	summarization + 文件 offloading + prompt caching
文件系统	直接读写宿主仓库	虚拟文件系统（StateBackend/StoreBackend/CompositeBackend）
权限控制	禁用外部 agent（粗粒度）	permissions 声明式 allow/deny（文件级）
数据隐私	无	PII middleware（redact/mask/hash/block）
模型降级	无	modelFallbackMiddleware
可观测性	tick JSON 自建	LangSmith 一等公民（每节点 trace + 评估）
生产验证	个人项目	LinkedIn/Uber/Klarna/Elastic
模型生态	Anthropic + OpenAI	任何 LangChain 兼容（含 Ollama 本地、GLM、通义）
平台依赖	必须挂 Claude Code/Codex/CodeBuddy	无平台依赖（纯 Python 进程）
软件开发专属能力	TDD REDGuard + 5 层验证 + 7 Gate	无（需自建）

三句话总结

- 1. **Deep Agents 有而 Auto-engineering 没有的**：sub-agent context 隔离、虚拟文件系统、context offloading、PII middleware、模型降级、LangSmith 可观测、模型无关、平台无关。**这些恰恰是银行生产场景更需要的。**
- 2. **Auto-engineering 有而 Deep Agents 没有的**：9 角色固定流程、TDD REDGuard、5 层验证、7+1 Gate、收敛判定。**这些是软件开发垂直场景专属，Deep Agents 要做软件开发 loop 必须自己补。**
- 3. **关键判断**：Auto-engineering 在"通用 harness"层是**重复造轮子**——它自建的 tick 编排、SQLite checkpoint、token 限流，Deep Agents 都有且更成熟。它真正的增量只在"软件开发垂直纪律"那一层。

六、10 项能力覆盖矩阵（六方）

把"软件开发 loop 的关键能力"拆成 10 项，看每个对象覆盖几项：

能力项	Claude Code (/goal、/loop)	Codex (/goal)	Superpowers	grill-me 1.1	Auto-engineering	LangGraph + Deep Agents
1. 需求澄清	✗	✗	✔	✔✔	✔	◐（需自建 supervisor）
2. 工作区隔离	✗	✗	✔	✗	✔	✔✔（sandbox 隔离容器）
3. 计划拆解	✔ Plan subagent	✔ Plan 阶段	✔	✗	✔	✔✔（task planning 内置）
4. TDD 红绿循环	✗	✗	✔	✗	✔✔	✗
5. 代码审查	✔ code-reviewer	✔ Review 阶段	✔	✗	✔✔	◐（supervisor pattern 自建）
6. 质量门禁	✗	✗	✗	✗	✔✔	◐（middleware 自建）
7. 收敛判定	◐ /goal Haiku 判	◐ Ralph Loop 自评	✗	✗	✔✔	◐（条件边自建）
8. 状态可恢复	✗ session-scoped	✔✔ app-server	✗	✗	✔✔	✔✔（Checkpointner + time travel）
9. token 成本控制	◐ /goal 较省	✔✔ budget_limited	✗	✗	✔✔	✔✔（limit middleware + caching + fallback）
10. 可审计性	✗	◐ app-server	✗	✗	✔✔	✔✔（LangSmith）

覆盖度统计（✔✔=2，✔=1，◐=0.5，✗=0）：

对象	覆盖度	定位
Auto-engineering	17/20	垂直最强
LangGraph + Deep Agents	15.5/20	通用最强
Superpowers	8/20	说服式全流程
Codex（含 /goal）	7.5/20	官方 loop 工程化最完整
Claude Code（含 /goal、/loop）	5/20	平台能力全但 loop 维度弱
grill-me 1.1	2/20	单点最强（需求澄清）

关键洞察：LangGraph + Deep Agents 以 15.5 分紧追 Auto-engineering 的 17 分，且它的"缺分项"（TDD 强制、5 层验证、7 Gate、9 角色）**全是软件开发垂直能力**——这些恰恰是银行团队自己补得上的（用 middleware + supervisor pattern）。反过来，Auto-engineering 缺的 sub-agent 隔离、虚拟文件系统、PII middleware、LangSmith 可观测、模型无关、平台无关，**补起来难得多。**

七、对银行的判断

7.1 六方对银行场景的适用性

对象	银行适用性	关键障碍
Claude Code (含 /goal、/loop)	• 仅非敏感场景	session-scoped 不可跑过夜；无强制门控；境外 API
Codex (含 /goal)	✗ 基本不可用	强制 ChatGPT 订阅，API key 不行；境外 API
Superpowers	• 可推广	零代码、用户量大，但境外 Claude Code 依赖
grill-me 1.1	✅ 可推广	3 句话、单点最强，需求澄清是银行项目最大浪费源头
Auto-engineering	• 范式值得学	个人项目无 SLA；只支持 Anthropic+OpenAI；平台依赖
LangGraph + Deep Agents	✅ 最值得认真评估	模型无关、平台无关、生产验证、银行刚需能力内置

7.2 LangGraph + Deep Agents 对银行的核心价值

- 1. 模型无关——支持 Ollama 本地、GLM、通义、文心，这是银行私有化部署的硬需求。Auto-engineering 只支持 Anthropic + OpenAI，银行基本不可用。
- 2. 平台无关——纯 Python 进程，不挂 Claude Code/Codex/CodeBuddy，不依赖境外 agent 平台。
- 3. 生产验证——LinkedIn/Uber/Klarna 在用，LangChain 官方维护。Auto-engineering 是个人项目无 SLA。
- 4. 银行刚需能力内置——PII middleware（数据脱敏）、permissions（文件级权限）、sandbox（代码执行隔离）、LangSmith（可观测审计）、modelFallback（模型降级）。这些 Auto-engineering 全没有。
- 5. 可观测性一等公民——LangSmith 每节点 trace，满足银行"每次模型调用留全量 prompt/response/版本/操作人/时间戳"的审计要求。Auto-engineering 的 tick JSON 是自建，远不如 LangSmith 成熟。

7.3 Auto-engineering 的真实定位

结合第二章项目发展史，Auto-engineering 的定位需要明确校准：

- 它不是"Python 强制式范式的代表"——LangGraph + Deep Agents 才是，且更成熟。
- 它是"在 Python 强制式范式内，针对软件开发场景的垂直纪律层"——这层有价值，但应该构建在 Deep Agents 之上，而不是从零自建 tick 编排。
- 它真正的可学习资产只剩两件：(1) “Python is the gatekeeper”原则；(2) 9 角色 + 7 Gate + TDD REDGuard 的软件开发纪律设计。其他（tick 编排、SQLite、token 限流、context 管理）Deep Agents 都有且更好。

7.4 银行落地路径

层	选型	理由
编排运行时	LangGraph	Pregel 超步 + checkpoint + time travel，生产验证
agent harness	Deep Agents	开箱即用 + sub-agent 隔离 + 虚拟文件系统 + PII middleware + LangSmith
LLM 后端	Ollama 本地 / 通义 / 文心 / GLM 私有化	模型无关，Deep Agents 原生支持
软件开发纪律	借鉴 Auto-engineering 的 9 角色 + 7 Gate + TDD REDGuard 设计，用 Deep Agents 的 middleware + supervisor pattern 实现	不 fork 它的代码，学它的纪律设计
合规编排层	银行自建	模型治理 + 决策留痕 + 可解释性 + 漂移检测
可观测	LangSmith（或自建等价物）	每节点 trace + 评估

7.5 分层组合 > 单体框架

综合六方对比，对银行 AI 应用建设给出最终结论：

场景	推荐组合	理由
个人/小团队提效	grill-me + Claude Code /goal	零代码、快速体会"AI 帮你想清楚 + 做完成"的价值
团队标准化开发	Superpowers + 自定义 SKILL.md	94 万用户验证，零代码、可快速推广
银行严肃场景 PoC	LangGraph + Deep Agents + 国产 LLM	模型无关、平台无关、生产验证、银行刚需能力内置
银行生产级 AI 应用	LangGraph + Deep Agents + 自建合规编排层 + 借鉴 Auto-engineering 纪律设计	分层组合，每层用最成熟的方案

核心判断：

- 说服式范式（Claude Code /goal、Codex /goal、Superpowers、grill-me）适合非敏感、轻量、快速场景。
- 强制式范式内，LangGraph + Deep Agents 是比 Auto-engineering 更优的底座——它更成熟、更通用、模型无关、平台无关、生产验证。
- Auto-engineering 的真正价值不在"通用 loop 框架"，而在"软件开发垂直纪律设计"——这部分值得借鉴，但应构建在 Deep Agents 之上，而非从零自建。
- 银行 AI 应用的护城河不在 loop 框架（这层会标准化），而在合规编排 + 数据边界 + 人在环这三块银行本来就比互联网公司熟的领域。

分层组合，每层用最成熟的方案，是六方对比后最明确的结论。

参考来源

LangGraph 相关

- [LangGraph Tutorial 2026: Build Stateful AI Agents for Enterprise — Alice Labs](#)

- [Design LangGraph — Pregel/BSP 超步执行模型](#)
- [Production-Ready Stateful AI Agents with Claude API + LangGraph — ClaudeLab](#)
- [LangGraph: 从链式调用到状态图——构建生产级 AI Agent 的图计算范式 — CSDN](#)
- [LangGraph + MCP: Multi-Agent Workflows \[2026 Guide\] — TechBytes](#)

Deep Agents 相关

- [Deep Agents overview — LangChain 官方文档](#)
- [Going to production with Deep Agents — LangChain 官方文档](#)
- [Backends \(StateBackend/StoreBackend/CompositeBackend/permissions\) — LangChain 文档](#)
- [deepagents 0.7.0a6 — PyPI \(26.3K★\)](#)
- [Introduction to LangChain Deep Agents and the Shift to ‘Agent 2.0’ — Spillwave](#)

框架对比相关

- [Agent 编排框架对比: LangGraph vs AutoGen vs CrewAI — CSDN](#)
- [Benchmarking Multi-Agent Frameworks: LangGraph Vs. Crewai Vs. Autogen — JATIR 2026](#)
- [LangGraph vs CrewAI vs AutoGen: Which AI Agent Framework Should You Use in 2026? — dev.to](#)
- [Agent Frameworks 2026: LangGraph vs CrewAI vs AutoGen vs OpenAI SDK — TokenMix](#)
- [CrewAI vs LangGraph vs AutoGen: 2026 Comparison — AgentCenter](#)

Claude Code / Codex /goal 相关

- [How to Use /goal and /loop in Claude Code for Autonomous Long-Running Workflows — MindStudio](#)
- [/goal による目標駆動実行 — はととはプロジェクト](#)
- [Claude Code v2.1.139 で追加された /goal コマンド — Developers.IO](#)
- [How to Use Claude Code’s /goal Command — MindStudio](#)
- [深入解析 Codex 智能体循环 — OpenAI](#)
- [How to Use OpenAI Codex’s /goal Command for Long-Running Autonomous Tasks — MindStudio](#)
- [Codex Goal mode 深度解析 — APIYI](#)
- [Codex CLI 0.128.0 Introduces /goal — AI Haberleri](#)

Superpowers / grill-me 相关

- [Superpowers — langlabs.io](#)
- [Superpowers — claude.com/plugins](#)
- [Claude Code Superpowers: Skills-framework — DataCamp](#)
- [Skills Changelog: Ubiquitous Language -> /grill-with-docs — AI Hero \(Matt Pocock\)](#)
- [grill-me — Agent Skills Finder](#)
- [Grill-Me and Writing Great Skills — DeepWiki](#)
- [The Claude Code Skill That Forces You to Think Before You Build — ClawdBytes](#)